# Génération de texte

Pour voir un exemple montrant comment bien démarrer avec Chat et le modèle Gemini Pro, exécutez le notebook "Commencer à utiliser Chat avec le modèle Gemini Pro" dans l'un des environnements suivants :

Ouvrir dans Colab (https://colab.research.google.com/github/GoogleCloudPlatform/generative-ai/blob/main/gemini/getting-started/intro_gemini_chat.ipynb?hl=fr)

|

Ouvrir dans Colab Enterprise (https://console.cloud.google.com/vertex-ai/colab/import/https%3A%2F%2Fraw.githubusercontent.com%2FGoogleCloudPlatform%2Fgenerative-ai%2Fmain%2Fgemini%2Fgetting-started%2Fintro_gemini_chat.ipynb?hl=fr)

|

Ouvrir dans Vertex AI Workbench (https://console.cloud.google.com/vertex-ai/workbench/deploy-notebook?download_url=https%3A%2F%2Fraw.githubusercontent.com%2FGoogleCloudPlatform%2Fgenerative-ai%2Fmain%2Fgemini%2Fgetting-started%2Fintro_gemini_chat.ipynb&hl=fr)

|

Afficher sur GitHub (https://github.com/GoogleCloudPlatform/generative-ai/blob/main/gemini/getting-started/intro_gemini_chat.ipynb)

Cette page vous explique comment envoyer des requêtes de chat au modèle Gemini à l'aide de la console Google Cloud , de l'API REST et des SDK compatibles.

Pour savoir comment ajouter des images et d'autres contenus multimédias à votre requête, consultez Compréhension des images (https://docs.cloud.google.com/vertex-ai/generative-ai/docs/multimodal/image-understanding?hl=fr).

Pour obtenir la liste des langues compatibles avec Gemini, consultez Langues acceptées (https://docs.cloud.google.com/vertex-ai/generative-ai/docs/learn/models?hl=fr#languages-gemini).

---

Pour explorer les modèles d'IA générative et les API disponibles sur Vertex AI, accédez à Model Garden dans la console Google Cloud .

**Accéder à la page "Jardin de modèles"** (https://console.cloud.google.com/vertex-ai/model-garden?hl=fr)

Si vous souhaitez utiliser Gemini directement à partir de vos applications mobiles et Web, consultez les SDK client Firebase AI Logic (https://firebase.google.com/docs/ai-logic?hl=fr) pour les applications Swift, Android, Web, Flutter et Unity.

## Générer du texte

Pour tester et effectuer des itérations de requêtes de chat, nous vous recommandons d'utiliser la consoleGoogle Cloud . Pour envoyer des requêtes de façon programmatique au modèle, vous pouvez vous servir de l'API REST, du SDK Google Gen AI, du SDK Vertex AI pour Python, ou de l'un des autres SDK et bibliothèques compatibles.

Vous pouvez utiliser des instructions système pour orienter le comportement du modèle en fonction d'un besoin ou d'un cas d'utilisation spécifiques. Par exemple, vous pouvez définir un persona ou un rôle pour un chatbot qui répond aux demandes adressées au service client. Pour en savoir plus, consultez les exemples de code d'instructions système (https://docs.cloud.google.com/vertex-ai/generative-ai/docs/learn/prompts/system-instructions?hl=fr#code_samples)
.

Vous pouvez utiliser le SDK Google Gen AI (https://docs.cloud.google.com/vertex-ai/generative-ai/docs/gemini-v2?hl=fr#google-gen) pour envoyer des requêtes si vous utilisez Gemini 2.0 Flash (https://docs.cloud.google.com/vertex-ai/generative-ai/docs/gemini-v2?hl=fr).

Voici un exemple simple de génération de texte.

Python Gen AI SDK | Go Gen AI SDK | Node.js Gen AI SDK | Java Gen AI SDK | C# Gen AI SDK

**Installer**

```
pip install --upgrade google-genai
```

Pour en savoir plus, consultez la documentation de référence du SDK (https://googleapis.github.io/python-genai/).

Définissez des variables d'environnement pour utiliser le SDK Gen AI avec Vertex AI :

```
# Replace the `GOOGLE_CLOUD_PROJECT` and `GOOGLE_CLOUD_LOCATION` values
# with appropriate values for your project.
export GOOGLE_CLOUD_PROJECT=GOOGLE_CLOUD_PROJECT
export GOOGLE_CLOUD_LOCATION=global
export GOOGLE_GENAI_USE_VERTEXAI=True
```

```
from google import genai
from google.genai.types import HttpOptions

client = genai.Client(http_options=HttpOptions(api_version="v1"))
response = client.models.generate_content(
    model="gemini-2.5-flash",
    contents="How does AI work?",
)
print(response.text)
# Example response:
# Okay, let's break down how AI works. It's a broad field, so I'll focus
#
# Here's a simplified overview:
# ...
```

## Réponses qui s'affichent progressivement ou en une fois

Vous pouvez choisir si le modèle génère des réponses qui s'affichent *progressivement* ou *en une fois*. Pour les réponses qui s'affichent progressivement, vous recevez chaque réponse dès que son jeton de sortie est généré. Pour les réponses qui s'affichent en une fois, vous recevez toutes les réponses lorsque l'ensemble des jetons de sortie sont générés.

Voici un exemple de génération de texte en streaming.

Python (#python)

Avant d'essayer cet exemple, suivez les instructions de configuration pour Python décrites dans le guide de démarrage rapide de Vertex AI sur l'utilisation des bibliothèques clientes (https://docs.cloud.google.com/vertex-ai/docs/start/client-libraries?hl=fr). Pour en savoir plus, consultez la documentation de référence de l'API Vertex AI pour Python (https://docs.cloud.google.com/python/docs/reference/aiplatform/latest?hl=fr).

Pour vous authentifier auprès de Vertex AI, configurez le service Identifiants par défaut de l'application. Pour en savoir plus, consultez Configurer l'authentification pour un environnement de développement local (https://docs.cloud.google.com/docs/authentication/set-up-adc-local-dev-environment?hl=fr).

```
from google import genai
from google.genai.types import HttpOptions

client = genai.Client(http_options=HttpOptions(api_version="v1"))
chat_session = client.chats.create(model="gemini-2.5-flash")

for chunk in chat_session.send_message_stream("Why is the sky blue?"):
    print(chunk.text, end="")
# Example response:
# The
#  sky appears blue due to a phenomenon called **Rayleigh scattering**. H
#  a breakdown of why:
# ...
```

## Comportement de Gemini dans les discussions multitours

Lorsque vous utilisez le chat multitours, Vertex AI stocke localement le contenu initial et les requêtes que vous avez envoyés au modèle. Vertex AI envoie toutes ces données à chaque requête ultérieure au modèle. Par conséquent, les coûts d'entrée pour chaque message que vous envoyez correspondent au total cumulé de toutes les données déjà envoyées au modèle. Si votre contenu initial est suffisamment volumineux, envisagez d'utiliser la mise en cache de contexte (https://docs.cloud.google.com/vertex-ai/generative-ai/docs/context-cache/context-cache-overview?hl=fr) lorsque vous créez l'objet de modèle initial pour mieux contrôler les coûts d'entrée.

## Étapes suivantes

- Découvrez comment envoyer des requêtes multimodales :

- Compréhension des images (https://docs.cloud.google.com/vertex-ai/generative-ai/docs/multimodal/image-understanding?hl=fr)
- Compréhension des vidéos (https://docs.cloud.google.com/vertex-ai/generative-ai/docs/multimodal/video-understanding?hl=fr)
- Compréhension audio (https://docs.cloud.google.com/vertex-ai/generative-ai/docs/multimodal/audio-understanding?hl=fr)
- Compréhension des documents (https://docs.cloud.google.com/vertex-ai/generative-ai/docs/multimodal/document-understanding?hl=fr)

- Découvrez les bonnes pratiques d'IA responsable et les filtres de sécurité de Vertex AI (https://docs.cloud.google.com/vertex-ai/generative-ai/docs/learn/responsible-ai?hl=fr).

Sauf indication contraire, le contenu de cette page est régi par une licence Creative Commons Attribution 4.0 (https://creativecommons.org/licenses/by/4.0/), et les échantillons de code sont régis par une licence Apache 2.0 (https://www.apache.org/licenses/LICENSE-2.0). Pour en savoir plus, consultez les Règles du site Google Developers (https://developers.google.com/site-policies?hl=fr). Java est une marque déposée d'Oracle et/ou de ses sociétés affiliées.

Dernière mise à jour le 2026/02/27 (UTC).